

FGV EPGE - Escola de Pós-Graduação em Economia

Mestrado Profissional em Economia

Decisões Baseadas em Dados I - 2025-4

Aluno: Jader Brenny Santana (jader_santana@hotmail.com)

Doutorando em Economia (EPGE), Mestre em Gestão de Tecnologia (TUHH-NIT, 2021) e Engenheiro Eletricista (UFPR, 2006).

Predição de Preços de Imóveis – Caixa Econômica Federal

Introdução

Este notebook desenvolve e avalia modelos de predição de preços de imóveis a partir de dados de leilões da Caixa Econômica Federal. O objetivo é comparar abordagens lineares e não lineares e analisar o papel de características físicas, locacionais e institucionais na explicação dos preços observados.

Dada a presença de não linearidades, alta cardinalidade categórica e heterogeneidade espacial, optou-se por modelos baseados em árvores, com destaque para o CatBoost. A análise distingue explicitamente entre modelos de natureza estrutural, voltados à interpretação econômica, e modelos estritamente preditivos, cujo foco é a maximização da acurácia.

Ao longo do notebook, são apresentados os dados, a construção dos cenários de modelagem, os resultados empíricos e sua interpretação.

Contexto institucional dos dados

Os dados utilizados referem-se a imóveis alienados pela Caixa Econômica Federal por meio de leilões e vendas online. Esses imóveis são, em sua maioria, ativos retomados pelo banco, frequentemente em decorrência de inadimplência, e disponibilizados ao público por meio de editais oficiais.

As vendas ocorrem em diferentes modalidades previstas em lei, com regras específicas de formação de preço. Observa-se empiricamente que as modalidades iniciais apresentam baixa taxa de arremate. Por esse motivo, a análise concentra-se nas

modalidades de Licitação Aberta e Venda Online, nas quais efetivamente se observa a maior parte das transações.

```
In [1]: from IPython.display import Image, display

display(Image(filename="timeline.jpg"))
```

• LINHA DO TEMPO:



Preparação e estrutura dos dados

Esta seção descreve a origem dos dados, a unidade de observação e os critérios adotados para seleção e organização das variáveis utilizadas nos modelos.

Dicionário de variáveis

A tabela a seguir apresenta as principais variáveis utilizadas na análise, agrupadas de acordo com sua natureza econômica e institucional. Foram consideradas exclusivamente informações disponíveis nos anúncios dos editais, isto é, anteriores ao processo de arrematação.

```
In [5]: import pandas as pd
import numpy as np

pathlocal = r"G:\My Drive\Github\py-2025-epge-dados1-finalassignment\data\input\
pathdrive = r"/content/drive/MyDrive/Github/py-2025-epge-dados1-finalassignment/

try:
    df = pd.read_parquet(pathlocal)
except:
    from google.colab import drive
    drive.mount("/content/drive")
    df = pd.read_parquet(pathdrive)

df.shape
```

```
Out[5]: (27928, 56)
```

```
In [6]: df = df.rename(columns={"preco": "lance_minimo"})
df = df.rename(columns={"valor_oferta": "valor_arrematado"})
df = df.rename(columns={"uf.x": "uf"})
df = df.rename(columns={"cidade.x": "cidade"})
df = df.rename(columns={"modalidade_de_venda.x": "modalidade_de_venda"})
```

```
In [7]: df_dicionario = pd.DataFrame([
    {"grupo": "Target", "variavel": "log_valor_arrematado", "descricao": "Log do"},

    {"grupo": "Locacional", "variavel": "uf", "descricao": "Estado do imóvel (Ex"},
    {"grupo": "Locacional", "variavel": "cidade", "descricao": "Município do imó"},
    {"grupo": "Locacional", "variavel": "bairro", "descricao": "Bairro do imóvel"},

    {"grupo": "Imóvel", "variavel": "area_privativa", "descricao": "Área privati"},
    {"grupo": "Imóvel", "variavel": "area_terreno", "descricao": "Área do terren"},
    {"grupo": "Imóvel", "variavel": "tipo_imovel", "descricao": "Tipo do imóvel"},

    {"grupo": "Leilão", "variavel": "modalidade_de_venda", "descricao": "Modalid"},

    {"grupo": "Financeiro", "variavel": "lance_minimo", "descricao": "Lance Míni"},
    {"grupo": "Financeiro", "variavel": "valor_avaliacao", "descricao": "Valor d"}
])
```

```
In [8]: # Passo 1 – Definir o target (y)

y = np.log(df["valor_arrematado"])

# Converter data_licitacao para datetime (tenta ISO e depois dia/mês/ano)
d = pd.to_datetime(df["data_licitacao"], errors="coerce")
if d.isna().mean() > 0.5:
    d = pd.to_datetime(df["data_licitacao"], errors="coerce", dayfirst=True)

df["data_licitacao_dt"] = d
df["ano"] = df["data_licitacao_dt"].dt.year
df["mes"] = df["data_licitacao_dt"].dt.month

cutoff_test = df["data_licitacao_dt"].max() - pd.DateOffset(months=6)
is_test = df["data_licitacao_dt"] >= cutoff_test

# Criar variáveis derivadas mínimas

df["bairro_std"] = df["bairro"].astype(str).str.upper().str.strip()
df.loc[df["bairro"].isna(), "bairro_std"] = "MISSING"

df["cidade_std"] = df["cidade"].astype(str).str.upper().str.strip()
df.loc[df["cidade"].isna(), "cidade_std"] = "MISSING"

df["uf_cidade"] = df["uf"].astype(str) + "_" + df["cidade_std"].astype(str)
df["uf_cidade_bairro"] = df["uf_cidade"] + "_" + df["bairro_std"]

# calcula frequências APENAS no treino
counts_train = df.loc[~is_test, "uf_cidade_bairro"].value_counts()
valid_train = counts_train[counts_train >= 15].index

# aplica a regra ao dataset inteiro (ok)
df["uf_cidade_bairro_f"] = np.where(
    df["uf_cidade_bairro"].isin(valid_train),
```

```

df["uf_cidade_bairro"],
    "OUTROS_BAIRROS"
)

count_cols = [ "area_total", "area_privativa", "area_terreno", "quartos", "salas",
               "lavabos", "suites", "cozinhas", "varandas", "sacadas", "terracos",
               "dce", "churrasqueiras", "wc", "wc_emp"
]

for c in count_cols:
    df[f"{c}_missing"] = df[c].isna().astype(int)

```

```

In [ ]: summary_ano = (
    df.groupby(["ano", "tipo", "modalidade_de_venda"])
      .agg(
        qtd_imoveis=("valor_arrematado", "count"),
        valor_total_em_milhoes_RS=("valor_arrematado", "sum"),
        valor_medio_em_milhoes_RS=("valor_arrematado", "mean"),
        valor_mediano_em_milhoes_RS=("valor_arrematado", "median")
      )
      .reset_index()
      .sort_values(["ano", "valor_total_em_milhoes_RS"], ascending=[True, False])
)

# Converter para milhões (numérico)
summary_ano["valor_total_em_milhoes_RS"] = summary_ano["valor_total_em_milhoes_RS"] / 1000000
summary_ano["valor_medio_em_milhoes_RS"] = summary_ano["valor_medio_em_milhoes_RS"] / 1000000
summary_ano["valor_mediano_em_milhoes_RS"] = summary_ano["valor_mediano_em_milhoes_RS"] / 1000000

summary_ano[["valor_total_em_milhoes_RS", "valor_medio_em_milhoes_RS", "valor_mediano_em_milhoes_RS"]]
summary_ano[["valor_total_em_milhoes_RS", "valor_medio_em_milhoes_RS", "valor_mediano_em_milhoes_RS"]]

```

```

In [10]: import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import LinearSegmentedColormap

# matriz tipo x modalidade
matriz = (
    summary_ano
      .pivot_table(
        index="tipo",
        columns="modalidade_de_venda",
        values="qtd_imoveis",
        aggfunc="sum",
        fill_value=0
      )
)

matriz = matriz.loc[matriz.sum(axis=1).sort_values(ascending=False).index, :]

# paleta: branco → verde escuro
cmap = LinearSegmentedColormap.from_list(
    "white_to_green",
    ["#ffffff", "#c7e9c0", "#41ab5d", "#006d2c"]
)

plt.figure(figsize=(5, 3)) # aumenta eixo X
img = plt.imshow(matriz.values, cmap=cmap, aspect="auto")

```

```

plt.colorbar(img, label="Quantidade de imóveis")

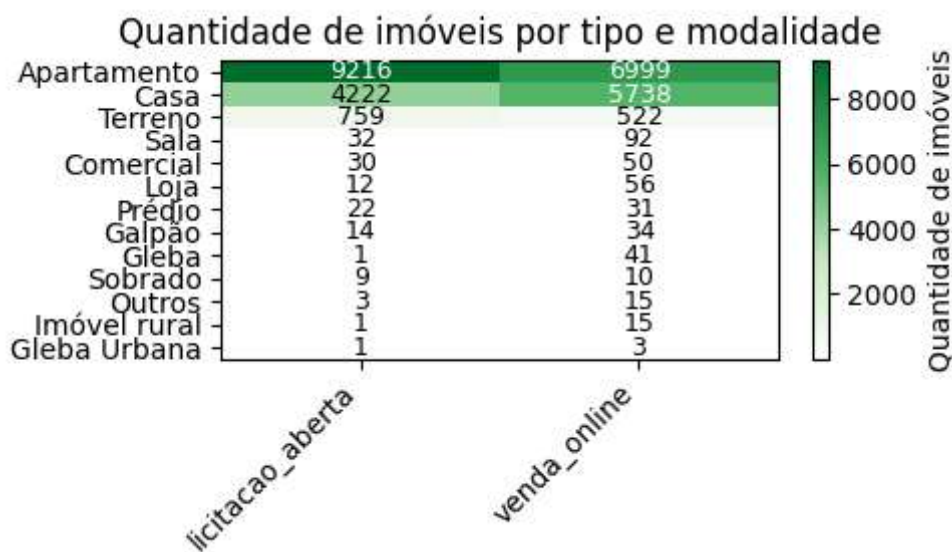
# eixos
plt.xticks(
    np.arange(len(matriz.columns)),
    matriz.columns,
    rotation=45,
    ha="right"
)

plt.yticks(
    np.arange(len(matriz.index)),
    matriz.index
)

# valores dentro das células
for i in range(matriz.shape[0]):
    for j in range(matriz.shape[1]):
        valor = matriz.values[i, j]
        plt.text(
            j, i, int(valor),
            ha="center", va="center",
            color="black" if valor < matriz.values.max() * 0.6 else "white",
            fontsize=9
        )

plt.title("Quantidade de imóveis por tipo e modalidade")
plt.tight_layout()
plt.show()

```



```

In [11]: import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import LinearSegmentedColormap

matriz = (
    df
    .pivot_table(
        index="ano",
        columns="modalidade_de_venda",
        values="valor_arrematado",
        aggfunc="sum",
        fill_value=0
    )
)

```

```

)

# ordenar linhas (maior em cima)
matriz = matriz.loc[matriz.sum(axis=1).sort_values(ascending=False).index, :]

cmap = LinearSegmentedColormap.from_list(
    "white_to_green",
    ["#ffffff", "#c7e9c0", "#41ab5d", "#006d2c"]
)

plt.figure(figsize=(5, 3))
img = plt.imshow(matriz.values, cmap=cmap, aspect="auto")
plt.colorbar(img, label="Valor total ofertado (R$)")

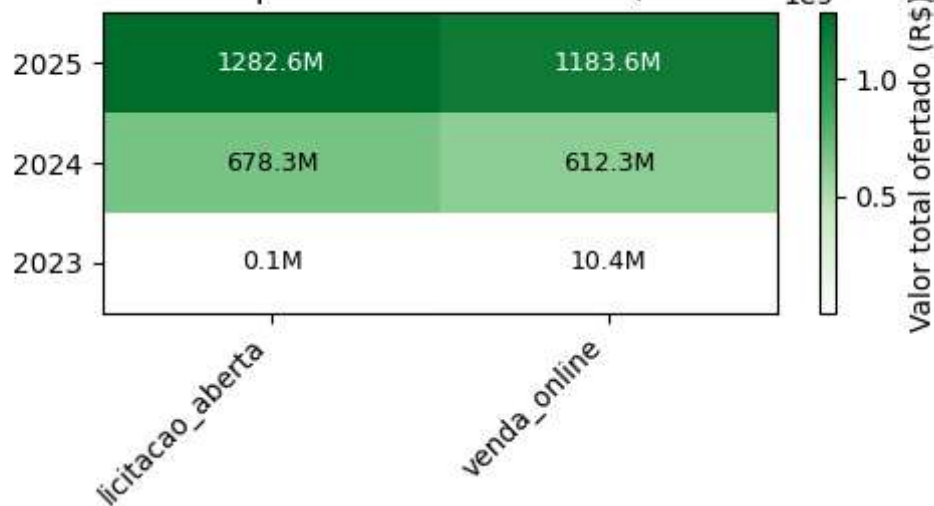
plt.xticks(np.arange(len(matriz.columns)), matriz.columns, rotation=45, ha="right")
plt.yticks(np.arange(len(matriz.index)), matriz.index)

# valores nas células (formatado em milhões, 1 casa)
for i in range(matriz.shape[0]):
    for j in range(matriz.shape[1]):
        v = matriz.values[i, j]
        txt = f"{v/1_000_000:.1f}M" if v != 0 else "0"
        plt.text(j, i, txt, ha="center", va="center", fontsize=9,
                 color="black" if v < matriz.values.max() * 0.6 else "white")

plt.title("Valor total ofertado por ano e modalidade (em milhões de R$)")
plt.tight_layout()
plt.show()

```

Valor total ofertado por ano e modalidade (em milhões de R\$)



```

In [12]: import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import LinearSegmentedColormap

# matriz tipo x modalidade
matriz = (
    summary_ano
    .pivot_table(
        index="ano",
        columns="modalidade_de_venda",
        values="qtd_imoveis",
        aggfunc="sum",
        fill_value=0
    )
)

```

```

    )
)

matriz = matriz.loc[matriz.sum(axis=1).sort_values(ascending=False).index, :]

# paleta: branco → verde escuro
cmap = LinearSegmentedColormap.from_list(
    "white_to_green",
    ["#ffffff", "#c7e9c0", "#41ab5d", "#006d2c"]
)

plt.figure(figsize=(5, 3)) # aumenta eixo X
img = plt.imshow(matriz.values, cmap=cmap, aspect="auto")

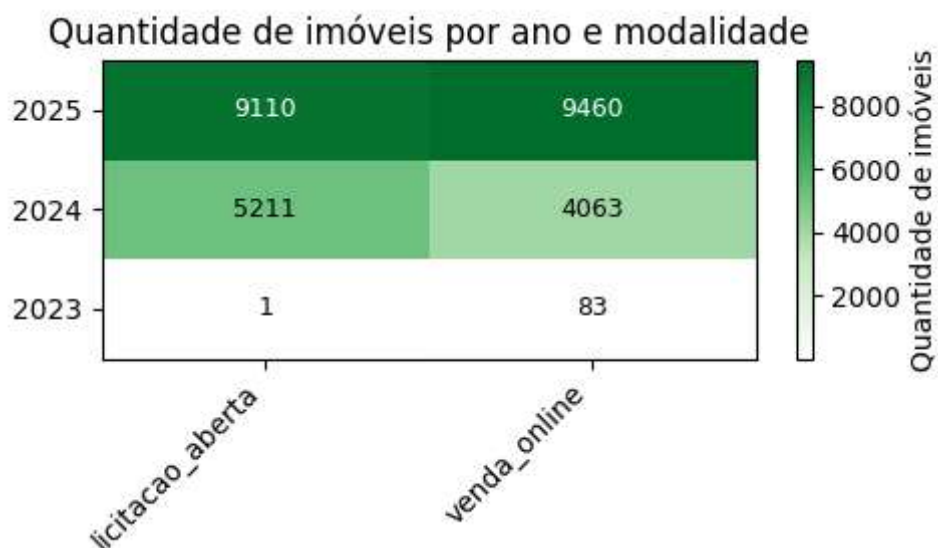
plt.colorbar(img, label="Quantidade de imóveis")

# eixos
plt.xticks(
    np.arange(len(matriz.columns)),
    matriz.columns,
    rotation=45,
    ha="right"
)
plt.yticks(
    np.arange(len(matriz.index)),
    matriz.index
)

# valores dentro das células
for i in range(matriz.shape[0]):
    for j in range(matriz.shape[1]):
        valor = matriz.values[i, j]
        plt.text(
            j, i, int(valor),
            ha="center", va="center",
            color="black" if valor < matriz.values.max() * 0.6 else "white",
            fontsize=9
        )

plt.title("Quantidade de imóveis por ano e modalidade")
plt.tight_layout()
plt.show()

```



Construção dos cenários de modelagem

Em vez de estimar um único modelo, optou-se por construir diferentes cenários de modelagem, variando o conjunto de variáveis explicativas. Essa estratégia permite avaliar como a inclusão de informações locais, financeiras e institucionais afeta o desempenho dos modelos.

Os cenários são organizados em dois grandes grupos:

- **Estruturais**, que excluem variáveis diretamente relacionadas a preços e têm foco interpretativo;
- **Preditivos**, que incorporam todas as informações disponíveis com o objetivo de maximizar a acurácia.

```
In [13]: # 2.2 Cenários de modelagem (somente mudando FEATURES e cat_cols)
```

```
BASE_CAT = ["modalidade_de_venda", "tipo"]
ESTADO = ["uf"]
CIDADE = ["uf_cidade"]
BAIRRO = ["uf_cidade_bairro_f"]

TIME = ["ano", "mes"]

BASE_NUM = [

    "area_total", "area_total_missing",
    "area_privativa", "area_privativa_missing",
    "area_terreno", "area_terreno_missing",

    "quartos", "quartos_missing",
    "salas", "salas_missing",
    "vagas_garagem", "vagas_garagem_missing",
    "lavabos", "lavabos_missing",
    "suítes", "suítes_missing",

    "cozinhas", "cozinhas_missing",
    "varandas", "varandas_missing",
    "sacadas", "sacadas_missing",
    "terraços", "terraços_missing",
    "áreas_serviço", "áreas_serviço_missing",
    "dce", "dce_missing",
    "churrasqueiras", "churrasqueiras_missing",
    "wc", "wc_missing",
    "wc_emp", "wc_emp_missing",
]

PRICE = ["lance_minimo"]
VALUATION = ["valor_de_avaliacao"]

SCENARIOS = {
    # modelo estrutural (sem âncoras monetárias)
    "structural_uf": {
        "modelselected": "CatBoostRegressor",
        "features": BASE_NUM + BASE_CAT + ESTADO + TIME,
        "cat_cols": BASE_CAT + ESTADO
    },
}
```



```

"structural_uf_city": {
  "modelselected": "CatBoostRegressor",
  "features": BASE_NUM + BASE_CAT + TIME + ESTADO + CIDADE,
  "cat_cols": BASE_CAT + ESTADO + CIDADE
},
"structural_uf_city_bairro": {
  "modelselected": "CatBoostRegressor",
  "features": BASE_NUM + BASE_CAT + TIME + ESTADO + CIDADE + BAIRRO,
  "cat_cols": BASE_CAT + ESTADO + CIDADE + BAIRRO
},
"structural_city": {
  "modelselected": "CatBoostRegressor",
  "features": BASE_NUM + BASE_CAT + TIME + CIDADE,
  "cat_cols": BASE_CAT + CIDADE
},
"structural_bairro": {
  "modelselected": "CatBoostRegressor",
  "features": BASE_NUM + BASE_CAT + TIME + BAIRRO,
  "cat_cols": BASE_CAT + BAIRRO
},
"valor_avaliacao_uf": {
  "modelselected": "CatBoostRegressor",
  "features": VALUATION + BASE_NUM + BASE_CAT + TIME + ESTADO,
  "cat_cols": BASE_CAT + ESTADO
},
"valor_avaliacao_uf_city": {
  "modelselected": "CatBoostRegressor",
  "features": VALUATION + BASE_NUM + BASE_CAT + TIME + ESTADO + CIDADE,
  "cat_cols": BASE_CAT + ESTADO + CIDADE
},
"valor_avaliacao_uf_city_bairro": {
  "modelselected": "CatBoostRegressor",
  "features": VALUATION + BASE_NUM + BASE_CAT + TIME + ESTADO + CIDADE + B
  "cat_cols": BASE_CAT + ESTADO + CIDADE + BAIRRO
},
"valor_avaliacao_city": {
  "modelselected": "CatBoostRegressor",
  "features": VALUATION + BASE_NUM + BASE_CAT + TIME + CIDADE,
  "cat_cols": BASE_CAT + CIDADE
},
"valor_avaliacao_bairro": {
  "modelselected": "CatBoostRegressor",
  "features": VALUATION + BASE_NUM + BASE_CAT + TIME + BAIRRO,
  "cat_cols": BASE_CAT + BAIRRO
},
# upper bound preditivo (com âncoras monetárias)
"predictive_uf": {
  "modelselected": "CatBoostRegressor",
  "features": PRICE + VALUATION + BASE_NUM + BASE_CAT + TIME + ESTADO,
  "cat_cols": BASE_CAT + ESTADO
},
"predictive_uf_city": {
  "modelselected": "CatBoostRegressor",
  "features": PRICE + VALUATION + BASE_NUM + BASE_CAT + TIME + ESTADO + CI
  "cat_cols": BASE_CAT + ESTADO + CIDADE
},
"predictive_uf_city_bairro": {
  "modelselected": "CatBoostRegressor",
  "features": PRICE + VALUATION + BASE_NUM + BASE_CAT + TIME + ESTADO + CI
  "cat_cols": BASE_CAT + ESTADO + CIDADE + BAIRRO
}

```

```

    },
    "predictive_city": {
        "modelselected": "CatBoostRegressor",
        "features": PRICE + VALUATION + BASE_NUM + BASE_CAT + TIME + CIDADE,
        "cat_cols": BASE_CAT + CIDADE
    },
    "predictive_bairro": {
        "modelselected": "CatBoostRegressor",
        "features": PRICE + VALUATION + BASE_NUM + BASE_CAT + TIME + BAIRRO,
        "cat_cols": BASE_CAT + BAIRRO
    },
}

ACTIVE_SCENARIO = "structural_uf" # <-- troque aqui quando quiser

FEATURES = SCENARIOS[ACTIVE_SCENARIO]["features"]
cat_cols = SCENARIOS[ACTIVE_SCENARIO]["cat_cols"]
modelselected = SCENARIOS[ACTIVE_SCENARIO]["modelselected"]

X = df[FEATURES].copy()
X.shape

```

Out[13]: (27928, 39)

Modelo baseline: regressão linear

Como referência inicial, estima-se um modelo baseline de regressão linear. Esse modelo tem caráter puramente comparativo e não busca capturar relações complexas ou não lineares.

Variáveis locais em níveis muito desagregados, como município e bairro, não possuem interpretação métrica e sua inclusão exigiria a criação de um grande número de dummies, resultando em um modelo pouco parcimonioso. Por esse motivo, o baseline linear utiliza uma representação espacial mais agregada.

```

In [14]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

def run_linear_baseline(df, features, cat_cols):
    X = df[features].copy()
    y = np.log(df["valor_arrematado"])

    X_train, X_test = X[~is_test], X[is_test]
    y_train, y_test = y[~is_test], y[is_test]

    from sklearn.impute import SimpleImputer

    num_cols = [c for c in features if c not in cat_cols]

    preprocessor = ColumnTransformer(
        transformers=[
            # categóricas: imputar missing -> depois one-hot
            ("cat", Pipeline(steps=[

```

```

        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ], cat_cols),

    # numéricas: imputar missing -> depois passa
    ("num", Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median"))
    ]), num_cols),
],
remainder="drop"
)

model = Pipeline([
    ("prep", preprocessor),
    ("lr", LinearRegression())
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

rmse = float(np.sqrt(mean_squared_error(y_test, y_pred)))
mae = float(mean_absolute_error(y_test, y_pred))
r2 = float(r2_score(y_test, y_pred))

return rmse, mae, r2

```

```

In [15]: linear_feats = BASE_CAT + BASE_NUM
linear_cats = BASE_CAT

rmse, mae, r2 = run_linear_baseline(df, linear_feats, linear_cats)

baseline_row = {
    "model": "LinearRegression",
    "scenario": "basic_linear",
    "rmse_log": rmse,
    "mae_log": mae,
    "r2": r2,
    "best_iteration": "n/a"
}

pd.DataFrame([baseline_row])

```

```

Out[15]:

```

	model	scenario	rmse_log	mae_log	r2	best_iteration
0	LinearRegression	basic_linear	0.574617	0.414028	0.236296	n/a

Modelos baseados em árvores (CatBoost)

Para lidar com não linearidades, interações complexas e variáveis categóricas de alta cardinalidade, utilizam-se modelos baseados em árvores, em particular o CatBoost. Essa classe de modelos permite incorporar variáveis locais e institucionais de forma flexível, sem a necessidade de codificação manual por dummies.

O código a seguir implementa um loop que estima automaticamente todos os cenários definidos anteriormente, utilizando o mesmo procedimento de treino, validação e avaliação.

In [16]: `!pip install catboost`

```
from catboost import CatBoostRegressor

def run_scenario(df, scenario_name, seed=42):
    feats = SCENARIOS[scenario_name]["features"]
    cats = SCENARIOS[scenario_name]["cat_cols"]

    X = df[feats].copy()
    y = np.log(df["valor_arrematado"])

    X_train_full = X.loc[~is_test].copy()
    X_test        = X.loc[is_test].copy()
    y_train_full = y.loc[~is_test].copy()
    y_test       = y.loc[is_test].copy()

    # =====
    # 2) Validação interna (últimos 20% do TREINO)
    # =====
    cutoff_val = df.loc[~is_test, "data_licitacao_dt"].quantile(0.8)
    is_val = df.loc[~is_test, "data_licitacao_dt"] >= cutoff_val

    X_train = X_train_full.loc[~is_val].copy()
    X_val    = X_train_full.loc[is_val].copy()
    y_train = y_train_full.loc[~is_val].copy()
    y_val    = y_train_full.loc[is_val].copy()

    assert set(X_train.index).isdisjoint(X_test.index)

    # =====
    # 3) Garantir categóricas como string
    # =====
    for c in cats:
        X_train[c] = X_train[c].astype(str)
        X_val[c]   = X_val[c].astype(str)
        X_test[c]  = X_test[c].astype(str)

    # =====
    # 4) Modelo
    # =====
    model = CatBoostRegressor(
        loss_function="RMSE",
        eval_metric="RMSE",
        iterations=2000,          # teto
        learning_rate=0.05,
```

```

depth=8,
random_seed=seed,
od_type="Iter",          # early stopping (overfitting detector)
od_wait=100,             # para se não melhorar por 100 iterações
verbose=False
)

model.fit(
X_train, y_train,
cat_features=cats,      # nomes das colunas categóricas
eval_set=(X_val, y_val),
use_best_model=True
)

best_iter = model.get_best_iteration()

# =====
# 5) Avaliação FINAL (teste intocado)
# =====
y_pred = model.predict(X_test)

rmse = float(np.sqrt(mean_squared_error(y_test, y_pred)))
mae = float(mean_absolute_error(y_test, y_pred))
r2 = float(r2_score(y_test, y_pred))

# =====
# 6) Importâncias
# =====
importances = model.get_feature_importance()
feat_names = X_train.columns

top = (
    pd.DataFrame({
        "feature": feat_names,
        "importance": importances
    })
    .sort_values("importance", ascending=False)
    .head(3)
    .reset_index(drop=True)
)

return rmse, mae, r2, top, best_iter

```

Requirement already satisfied: catboost in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (1.2.8)

Requirement already satisfied: graphviz in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from catboost) (0.21)

Requirement already satisfied: matplotlib in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from catboost) (3.10.0)

Requirement already satisfied: numpy<3.0,>=1.16.0 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from catboost) (2.1.3)

Requirement already satisfied: pandas>=0.24 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from catboost) (2.3.1)

Requirement already satisfied: scipy in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from catboost) (1.15.3)

Requirement already satisfied: plotly in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from catboost) (6.3.0)

Requirement already satisfied: six in c:\users\jader\appdata\roaming\python\python312\site-packages (from catboost) (1.17.0)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\jader\appdata\roaming\python\python312\site-packages (from pandas>=0.24->catboost) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from pandas>=0.24->catboost) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from pandas>=0.24->catboost) (2025.2)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from matplotlib->catboost) (1.3.3)

Requirement already satisfied: cycler>=0.10 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from matplotlib->catboost) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from matplotlib->catboost) (4.59.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from matplotlib->catboost) (1.4.8)

Requirement already satisfied: packaging>=20.0 in c:\users\jader\appdata\roaming\python\python312\site-packages (from matplotlib->catboost) (25.0)

Requirement already satisfied: pillow>=8 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from matplotlib->catboost) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from matplotlib->catboost) (3.2.3)

Requirement already satisfied: narwhals>=1.15.1 in c:\users\jader\appdata\local\programs\python\python312\lib\site-packages (from plotly->catboost) (2.1.1)

Resultados

A tabela a seguir resume o desempenho dos diferentes cenários de modelagem, permitindo comparar diretamente os ganhos obtidos com a introdução de modelos não lineares e diferentes níveis de granularidade espacial.

```
In [17]: rows = []

for name in SCENARIOS.keys():
    rmse, mae, r2, top, best_iter = run_scenario(df, name)

    row = {
        "model": "CatBoost",
        "scenario": name,
        "rmse_log": rmse,
        "mae_log": mae,
        "r2": r2,
        "best_iteration": best_iter
    }
```

```

for i in range(len(top)):
    row[f"top{i+1}_f"] = top.loc[i, "feature"]
    row[f"top{i+1}_i"] = top.loc[i, "importance"]

    rows.append(row)

results = pd.DataFrame(rows)

results = pd.concat(
    [pd.DataFrame([baseline_row]), results],
    ignore_index=True
)

results

```

Out[17]:

	model	scenario	rmse_log	mae_log	r2	best_iter
0	LinearRegression	basic_linear	0.574617	0.414028	0.236296	
1	CatBoost	structural_uf	0.470362	0.344461	0.488279	
2	CatBoost	structural_uf_city	0.425640	0.306825	0.580962	
3	CatBoost	structural_uf_city_bairro	0.421009	0.300449	0.590031	
4	CatBoost	structural_city	0.444300	0.322391	0.543416	
5	CatBoost	structural_bairro	0.513106	0.381840	0.391047	
6	CatBoost	valor_avaliacao_uf	0.302975	0.194979	0.787684	
7	CatBoost	valor_avaliacao_uf_city	0.293447	0.189315	0.800828	
8	CatBoost	valor_avaliacao_uf_city_bairro	0.294186	0.189391	0.799823	
9	CatBoost	valor_avaliacao_city	0.299679	0.192615	0.792278	
10	CatBoost	valor_avaliacao_bairro	0.306985	0.197178	0.782027	
11	CatBoost	predictive_uf	0.208553	0.141590	0.899399	
12	CatBoost	predictive_uf_city	0.197632	0.133593	0.909659	
13	CatBoost	predictive_uf_city_bairro	0.200846	0.135817	0.906697	
14	CatBoost	predictive_city	0.205060	0.139670	0.902741	
15	CatBoost	predictive_bairro	0.206782	0.142418	0.901100	

In []:

Resultados e interpretação

Os resultados evidenciam ganhos substanciais de desempenho ao se substituir o modelo linear baseline por modelos baseados em árvores. Enquanto a regressão linear apresenta baixo poder explicativo, os modelos não lineares capturam de forma mais adequada a relação entre características físicas, locais e institucionais dos imóveis e seus preços.

Nos modelos estruturais, observa-se que a localização é o principal determinante do preço, com ganhos relevantes à medida que se aumenta a granularidade espacial. A combinação de níveis hierárquicos (UF e município) melhora o desempenho em relação ao uso isolado de um único nível, ainda que essa informação seja redundante em termos estritos. Esse resultado sugere que variáveis hierárquicas atuam como um mecanismo de regularização no aprendizado dos modelos baseados em árvores.

A inclusão de variáveis financeiras, em especial o valor de avaliação divulgado pela Caixa, produz um salto expressivo de desempenho. Nesse caso, a granularidade espacial torna-se menos relevante, indicando que o modelo passa a capturar principalmente a relação entre valor de avaliação e preço com valor de oferta (valor de arremate). Esses modelos devem ser interpretados como ferramentas preditivas, e não como modelos estruturais de formação de preços.

De forma geral, os resultados reforçam a distinção entre modelos com foco interpretativo e modelos voltados à maximização da acurácia, bem como a importância de alinhar a especificação do modelo ao objetivo da análise.

Extensões futuras

Como extensão natural deste trabalho, pretende-se incorporar coordenadas geográficas e técnicas de clusterização espacial, como o HDBSCAN, para capturar heterogeneidade intraurbana de forma contínua, reduzindo a dependência de divisões administrativas tradicionais.

Referências

CAIXA ECONÔMICA FEDERAL. Venda de imóveis CAIXA. Brasília: CAIXA, s.d. Disponível em: <https://www.caixa.gov.br/voce/habitacao/imoveis-venda/Paginas/default.aspx> . Acesso em: 01 dez. 2025.

PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, v. 12, p. 2825–2830, 2011. Disponível em: <https://scikit-learn.org/stable/> . Acesso em: 01 dez. 2025.

GÉRON, A. Hands-on machine learning with Scikit-Learn, Keras & TensorFlow. 3. ed. Sebastopol: O'Reilly Media, 2022.

OPENAI. ChatGPT: Large language model for code generation and analytical support. San Francisco: OpenAI, s.d. Disponível em: <https://openai.com/chatgpt> . Acesso em: 01 dez. 2025.